

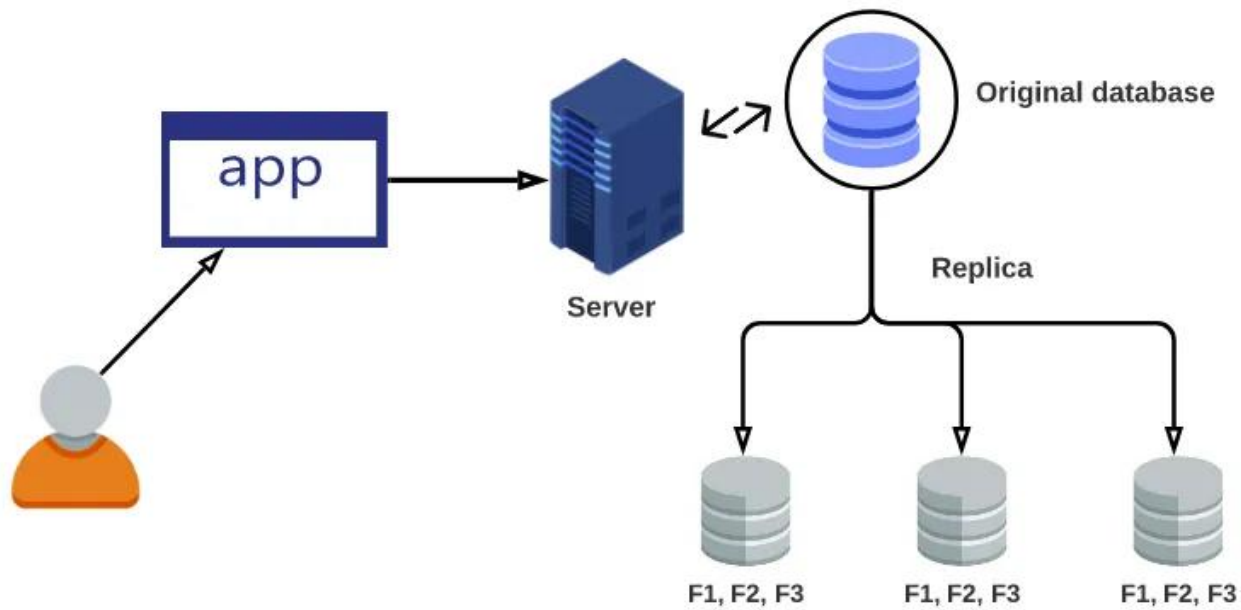
Data Replication & Consistency Models in Distributed Systems

Lecture 6

What is Data Replication?

- Data replication is the process of storing copies of the same data on multiple nodes to improve availability, fault tolerance, and performance.
- Key Points:
 - Ensures data availability during failures.
 - Reduces read latency.
 - Helps distribute load across multiple servers.

Full replication in DBMS



[F1, F2, F3 are the different fragments of the main database]

Why is Replication Important?

- Data replication is essential in distributed systems for several reasons:
 1. High Availability - If one server goes down, another replica can serve requests.
 2. Fault Tolerance - Systems can recover from hardware or network failures.
 3. Load Balancing - Distributes traffic across multiple servers to avoid bottlenecks.
 4. Low Latency - Data can be stored closer to users, reducing response time.
- Real-World Example: Netflix uses replication to stream content efficiently worldwide.

Challenges in Replication

- While replication offers many advantages, it also comes with challenges:
 1. Consistency Issues - Keeping all copies of data synchronized.
 2. Increased Storage Costs - Replicating large datasets requires more storage space.
 3. Network Overhead - Synchronizing replicas over a network can introduce delays.
 4. Concurrency Control - Handling multiple updates to the same data from different nodes.

Types of Data Replication

- Replication techniques determine how data is copied across distributed systems.
- Main Types:
 1. Active Replication - All replicas execute requests simultaneously.
 2. Passive Replication - A primary node processes requests and updates backups.
 3. Quorum-Based Replication - Read/write operations require a subset (quorum) of replicas to agree.
- Choosing a method depends on consistency, availability, and network overhead.

Active Replication (Synchronous Replication)

- In active replication, all replicas receive and process the same request simultaneously.
- Example: Used in high-availability financial trading systems.

Pros:

- High availability and fault tolerance.
- Consistent state across all replicas.

Cons:

- High network and computation overhead.
- Requires strict coordination.
- Not ideal for geographically distributed systems.

Passive Replication (Primary-Backup Model)

- A primary (leader) node handles requests and synchronizes state with backup replicas.

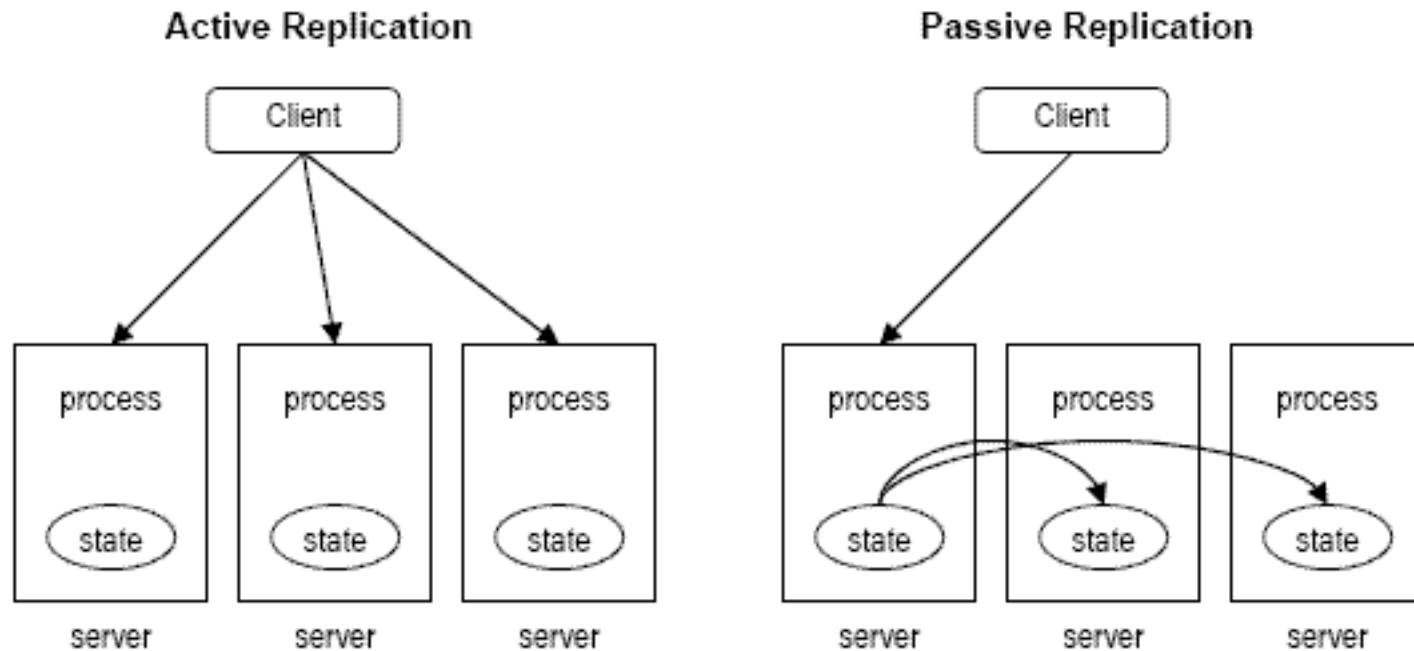
Pros:

- More efficient than active replication.
- Works well for read-heavy workloads.

Cons:

- Risk of stale reads if backups lag behind.
 - Primary failure requires leader election (downtime risk).
-
- Example: MySQL Replication, PostgreSQL Streaming Replication.

Active vs Passive Replication



Quorum-Based Replication

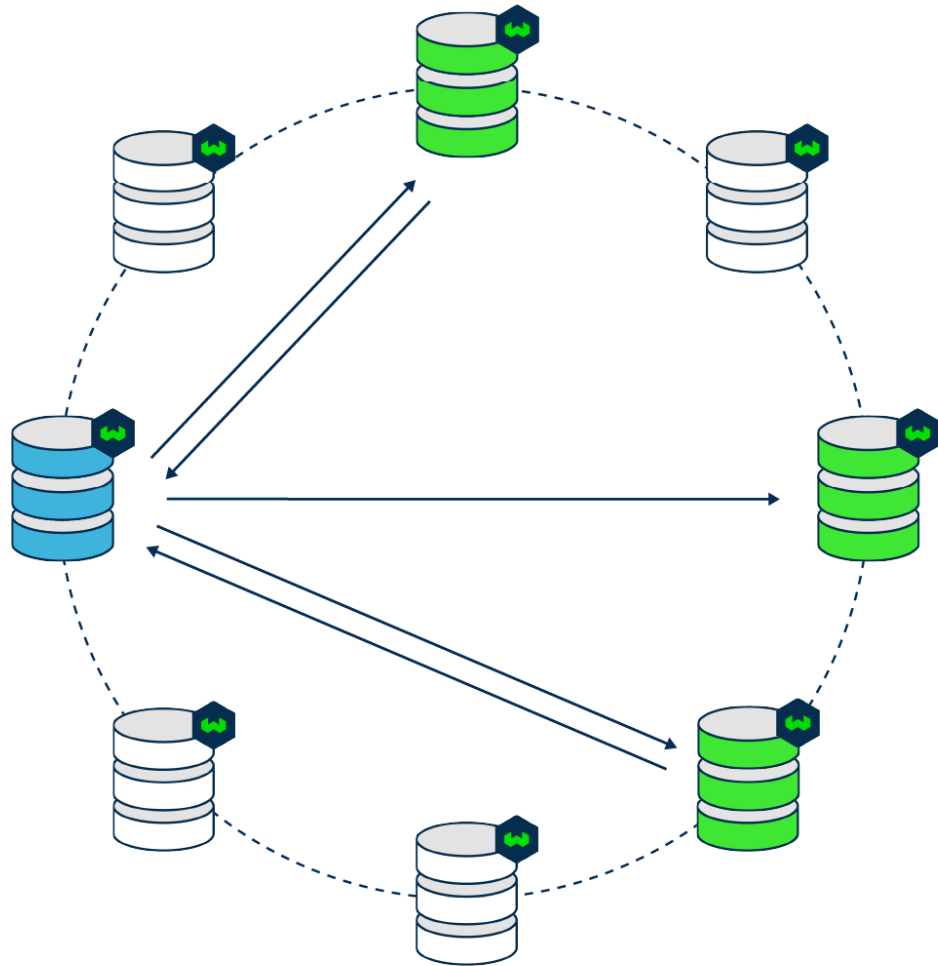
- Instead of requiring updates on all replicas, quorum-based replication allows a subset of replicas to agree on operations.
-  Balances consistency and availability.
-  Supports distributed systems with network partitions.
-  Requires careful tuning of read/write quorum sizes.
- Example: Used in Apache Cassandra and Amazon DynamoDB.

**Consistency level =
QUORUM**

Replication Factor = 3



Client



Example: Quorum Replication in DynamoDB and Cassandra

- Read Quorum (R): Minimum replicas that must return the latest data.
- Write Quorum (W): Minimum replicas that must confirm a write before success.
- Rule: $R + W > \text{Total replicas}$ ensures strong consistency.

Example: Quorum-Based Replication

- A Cassandra cluster has 7 nodes ($N=7$). If the system wants to ensure strong consistency,
- What could be possible values for R (read quorum) and W (write quorum) based on the formula $R + W > N$?
- What if $R + W = N$?

Example: Quorum-Based Replication

- Possible solutions:
 - (R=4, W=4) → Ensures strong consistency (since $4+4 > 7$).
 - (R=3, W=5) → Also valid since $3+5 > 7$, but allows more availability for reads.
 - (R=5, W=3) → Prioritizes consistency of reads.
- Different configurations optimize for consistency, availability, or performance.

Example: Quorum-Based Replication

- $R + W = N \rightarrow$ Eventual Consistency
- $(R = 3, W = 4) \rightarrow$ Some reads may not get the latest write

Comparison of Replication Strategies

Replication Type	Consistency	Availability	Overhead
Active	Strong	Low	High
Passive	Moderate	High	Medium
Quorum	Adjustable	High	Medium

Replication Topologies

- Defines how data is distributed and synchronized between replicas (Architecture).
- Impacts system performance, fault tolerance, and scalability.
- Choice depends on consistency, availability, and network constraints.
- Determines how failures are handled.
- Affects read and write performance.
- Plays a crucial role in distributed databases.

Master-Slave Replication

- One master node handles all writes.
- Primarily used in passive replication.
- Slave nodes replicate data from the master asynchronously or synchronously.
- Read requests can be served by slaves to reduce load on the master.
-  Advantages: Improves read performance, simplifies consistency.
-  Disadvantages: Single point of failure, write bottleneck, complex failover.
-  Example Systems: MySQL, PostgreSQL (Primary-Replica Mode).

Multi-Master Replication

- Multiple master nodes accept both read and write operations.
- Changes must be synchronized between all masters.
- Primarily used in active replication
-  Advantages: No single point of failure, higher write availability.
-  Disadvantages: Conflict resolution required, higher synchronization overhead.
-  Example Systems: Google Cloud Spanner, CouchDB, Firebase Firestore.

Ring Replication

- Data is replicated across nodes in a ring-like structure.
- Updates propagate in a circular manner.
- Often used in peer-to-peer distributed databases.
- More suitable for passive replication
-  Advantages: Efficient for distributed writes, fault tolerance.
-  Disadvantages: Higher read latency, complex consistency management.
-  Example Systems: Apache Cassandra (Dynamo-style ring architecture).

Replication Latency

- **Replication latency** is the **time delay between when data is written to one replica and when that update is propagated and visible on other replicas** in a distributed system.
- Replication Latency = Data Size/Replication Speed + Network Delay + Processing Time
- Real-World Relevance:
 - Impacts consistency in geo-replicated databases like Google Spanner.
 - Determines trade-offs between latency and correctness.

Example: Replication Latency Calculation

- A system replicates a 1MB file with a network delay of 200ms and processing time of 100ms. If the replication speed is 10MB/s, how long will replication take?
 - Use the formula: Replication Latency = (Data Size/Replication Speed + Network Delay + Processing Time

Answer: Replication Latency Calculation

- Solution:
 - Replication Latency = $(1\text{MB} / 10\text{MBps}) + 200\text{ms} + 100\text{ms}$
 - $= 0.1\text{s} + 0.2\text{s} + 0.1\text{s} = 0.4\text{s} (400\text{ms})$

Hint: How can pipelining improve this?

What is Consistency in Distributed Systems?

- Consistency ensures that all nodes in a distributed system have a uniform view of data.
- Key Points:
 - Why Consistency Matters: Prevents stale or conflicting data across nodes.
 - Trade-offs: Strong consistency ensures correctness, but can reduce availability.
 - Connection to Replication: Consistency defines how updates propagate across replicas.

CAP Theorem review

- The CAP theorem states that a distributed system can achieve at most two of the following properties:
 1. Consistency (C): Every read receives the latest write.
 2. Availability (A): System always responds, even during failures.
 3. Partition Tolerance (P): System continues to operate despite network failures.
- Example:
 - Google Spanner (CP: Strong Consistency, Partition Tolerance)
 - DynamoDB (AP: High Availability, Partition Tolerance)

Stale Reads vs. Failed Requests

- Stale Read: A read operation that returns outdated (old) data instead of the latest version.
 - Example: Seeing an older version of a post on a social media app due to replication lag.
- Failed Request: A request that does not return any response (timeout, server failure, or rejection).
 - Example: A banking transaction fails because the database is temporarily unavailable.
- Key Difference: A system can return stale data (low consistency) while still being highly available.

Consistency vs Availability

- Consistency (C):

$$C = 1 - (\text{Stale Reads} / \text{Total Reads})$$

- Measures how often data is fresh.
- A high stale read ratio means lower consistency.

- Availability (A):

$$A = 1 - (\text{Failed Requests} / \text{Total Requests})$$

- Measures how often the system successfully responds.
- A high failure rate means lower availability.

- Key Insight: A system can have 100% availability but still return stale data.
- Consistency and Availability can be inversely proportional. High consistency may lead to low availability and vice versa

Strong Consistency (Linearizability)

- A system is strongly consistent if every read reflects the most recent write.
-  Guarantees correctness.
-  High network overhead, lower availability.
- Use Cases:
 - Banking transactions.
 - Stock trading platforms.
 - Distributed databases requiring strict consistency.

Eventual Consistency

- Eventual consistency guarantees that all replicas will converge to the same state over time, but not immediately.



High availability and low latency.



Temporary inconsistencies between replicas.

- Use Cases:
 - Social media feeds (Facebook, Twitter)
 - DNS systems
 - Caching layers (CDNs)

Why Some Systems Prefer Eventual Consistency

- Ensures high availability and scalability.
- Used in social media, caching layers, DNS.
- Tolerates temporary inconsistencies.
- Example: Facebook posts appearing out of order temporarily.

Eventual Consistency Convergence Time

- Formula:

$$T_{\text{converge}} = (\text{Number of Replicas} \times \text{Update Size}) / \text{Replication Bandwidth}$$

- Real-World Relevance:

- Important in AP systems like Amazon DynamoDB and Facebook TAO.

- Measures how fast data consistency is achieved globally.

- Example: A 1MB update across 10 replicas with 50Mbps bandwidth takes ~1.6s to converge.

Example: Eventual Consistency

Convergence Time

- A system has 5 replicas, each receiving updates of 2MB.
- The replication bandwidth is 50Mbps.
- How long will it take for all replicas to converge?
- $T_{\text{converge}} = (\text{Number of Replicas} \times \text{Update Size}) / \text{Replication Bandwidth}$

Answer: Eventual Consistency

Convergence Time

- Formula: $T_{\text{converge}} = (\text{Number of Replicas} \times \text{Update Size}) / \text{Replication Bandwidth}$
- Step 1: Convert 50Mbps to MBps $\rightarrow 50 / 8 = 6.25\text{MBps}$
- Step 2: Compute: $(5 \times 2\text{MB}) / 6.25\text{MBps} = 10 / 6.25 = 1.6 \text{ seconds}$

Probability of Stale Reads

- Formula:

$$P_{\text{stale}} = 1 - e^{-\lambda T}$$

- Adopted from Poisson Distribution
- Real-World Relevance:
 - Measures the likelihood of reading outdated data in eventual consistency systems.
 - Used in NoSQL databases like DynamoDB and Cosmos DB.

Modeling DS events with Poisson Distribution

- Why Poisson is used in distributed systems
 - Random arrival of writes
 - Independent events
 - Stable average rate (λ)

Poisson Process Assumptions

1. Events are Random (Not Scheduled)
 - Writes occur irregularly and bursty
 - 2. Events are Independent
 - One write does not force or prevent another
 - 3. Constant Average Rate (λ)
 - Long-term stable average (e.g., 3 writes/sec)

Poisson Probability Formula

- Number of writes in time T : $N(T) \sim \text{Poisson}(\lambda T)$
 - Probability of exactly k writes:

$$P(k) = \frac{(\lambda T)^k e^{-\lambda T}}{k!}$$

Probability of Fresh Data

- Fresh = No write during delay T
 - $P(\text{fresh}) = P(0 \text{ writes})$
 - $P(\text{fresh}) = e^{-\lambda T}$

Probability of Stale Data

- Stale = At least one write occurred
 - $P(\text{stale}) = 1 - P(0)$
 - $P(\text{stale}) = 1 - e^{-\lambda T}$

Intuition Check

- If λ increases (more writes) $\rightarrow$ Fresh probability decreases
 - If T increases (longer delay) $\rightarrow$ Fresh probability decreases
 - Therefore, Staleness increases with λ and T

Example: Probability of Stale Reads

- A system updates data at a rate of 3 updates/sec.
- If a read request experiences a 2-second delay, what is the probability that it reads stale data?
- $P_{\text{stale}} = 1 - e^{-\lambda T}$

Answer: Probability of Stale Reads

- Formula: $P_{\text{stale}} = 1 - e^{-\lambda T}$
- Step 1: $\lambda = 3$ updates/sec, $T = 2$ sec
- Step 2: Compute: $P_{\text{stale}} = 1 - e^{(-3 \times 2)} = 1 - e^{(-6)} \approx 1 - 0.00247 \approx 99.75\%$
- This means there is a 99.75% chance of reading stale data.

Conflict Resolution in Eventual Consistency

- Why do conflicts occur in eventually consistent systems?
- Multiple replicas may temporarily diverge.
- Conflicts arise due to concurrent writes, network partitions, and replica synchronization delays.
- Solution: Use conflict resolution techniques to reconcile different versions of data.

Vector Clocks – Tracking Causal History

- Vector clocks track the order of events in distributed systems.
- Each replica maintains a vector of counters to record updates.
-  Advantages: Provides precise tracking of causality between events.
-  Disadvantages: Vector sizes grow with nodes, computational overhead.
-  Example Systems: Amazon DynamoDB, Riak, Apache Cassandra.

Using Vector clocks to detect conflicts

Given two vectors **A** and **B**:

- If $A \leq B$ for all entries and $<$ for at least one $\rightarrow$ A happened before B
- If $B \leq A$ for all entries and $<$ for at least one $\rightarrow$ B happened before A
- Otherwise $\rightarrow$ Concurrent $\rightarrow$ Conflict

Vector Clocks – Conflict resolution example

Vector Clock Updates for Transactions

1. Initial state: $[1,1]$ (Both nodes see 500K balance).
2. Colombo ATM Server processes the 300K withdrawal $\rightarrow$ Updates clock to $[2,1]$.
3. Kandy ATM Server processes the 250K withdrawal $\rightarrow$ Updates clock to $[1,2]$.
4. When nodes synchronize, they detect conflicting transactions ($[2,1]$ vs. $[1,2]$), indicating concurrent operations.
5. Vector Clocks only detect conflicts

Last-Write-Wins (LWW) – Simplicity Over Accuracy

- LWW assigns timestamps to updates and keeps the most recent one.
- Ensures that the latest update wins, discarding previous conflicting updates.
-  Advantages: Simple, efficient conflict resolution.
-  Disadvantages: Risk of data loss, timestamp skew issues.
-  Example Systems: Apache Cassandra, Redis, Amazon S3.

LWW Example

Scenario:

- A user edits their **Facebook post** on two different devices at the same time.
- Each device sends an update to different database replicas.
- The system uses **Last-Write-Wins (LWW)** to resolve conflicts.

CRDTs – Conflict-Free Replicated Data Types

- CRDTs ensure eventual consistency without conflicts using special data structures.
- Operations can be applied in any order and still produce the correct result.
-  Advantages: No need for conflict resolution logic, merges changes automatically.
-  Disadvantages: More complex data structures, higher memory usage.
-  Example Systems: Riak, AntidoteDB, Redis CRDTs.

What is a CRDT?

- Distributed data structure for replication
- Ensures eventual consistency
- Automatically merges concurrent updates
- No coordination required

Core Properties

- Operations must be:
- Commutative (order doesn't matter)
- Associative (grouping doesn't matter)
- Idempotent (duplicates don't matter)

Example: Grow-Only Counter (G-Counter)

- Used for likes, views, downloads
- Each replica keeps local counter
- Merge = max per replica

G-Counter Scenario

- Replica A increments 3
- Replica B increments 2
- Merge $\rightarrow [3,2]$
- Total = 5

Why This Works

- Order doesn't matter
- A then $B = 5$
- B then $A = 5$
- Always converges

Example: Collaborative Editing

- User A types 'Hello'
- User B types 'World'
- After sync both replicas agree

Why No Conflict Occurs

- CRDT tracks operations
- Not just final values
- System merges updates automatically

Example: Distributed Shopping Cart

- Replica A adds Laptop
- Replica B adds Mouse
- After sync $\rightarrow$ {Laptop, Mouse}

Normal Replication vs CRDT

- Normal: overwrites possible
- CRDT: automatic merge
- Normal: order matters
- CRDT: order irrelevant

Databases Using CRDTs

- Riak
 - Distributed key-value store
 - Uses CRDT sets, counters, maps
- Redis CRDT
 - Active-active geo replication
 - Conflict-free multi-region writes
- AntidoteDB
 - Geo-replicated database
 - Uses CRDTs + causal consistency

Applications Using CRDTs

- Collaborative editors
 - Simultaneous editing
- Shared whiteboards
 - Multiple users drawing together
- Multiplayer games
 - Consistent shared state
- Offline-first apps
 - Sync after reconnection

Key Takeaway

- CRDTs eliminate conflicts by design
- All replicas converge automatically

Choosing the Right Conflict Resolution Method

- Vector Clocks → When full event order history is needed (e.g., banking transactions). Only conflict detection
- Last-Write-Wins → When losing some updates is acceptable (e.g., social media feeds).
- CRDTs → When automatic merging is needed (e.g., collaborative editing tools).
- Application-defined merge logic (e.g. reject last update, rollbacks)
- Majority voting (e.g. choose value held by majority)
- Etc.
- Modern distributed systems use a mix of these techniques based on requirements.

Causal Consistency

- Causal consistency ensures that operations with causal dependencies are seen in the correct order, but concurrent updates may appear out of order.
-  Balances consistency and performance.
-  Not as strict as strong consistency.
- Use Cases:
 - Collaborative document editing (Google Docs).
 - Distributed chat systems (Slack, WhatsApp).

Example: Causal Consistency in Social Media Systems

- Social media platforms use causal consistency to maintain logical ordering of posts and comments.
- Example: Facebook
 - If Alice likes Bob's post before Charlie sees it, Charlie must see the like before adding a comment.
-  Ensures logical flow of interactions.
-  Not as strict as strong consistency.

Session Consistency – Making Reads More Predictable

- Ensures that a single client's reads and writes remain consistent within a session.
- The client always sees its own latest writes, but other clients may see older values.
-  Advantages: Reduces staleness of reads, improves usability.
-  Disadvantages: No guarantees for other clients, requires tracking sessions.
-  Example Systems: Amazon DynamoDB, Google Cloud Firestore.

Session consistency - example

Example: Online Shopping Cart (E-Commerce)



Scenario:

- Alice logs into an **e-commerce website** (Amazon, eBay).
- She **adds a laptop to her cart** from her **mobile phone**.
- She then **switches to her laptop** and continues shopping.

Read-Your-Writes Consistency

- Ensures that after a client writes data, any subsequent reads from that client reflect the update.
- Prevents the issue where a client writes data but immediately reads an old version.
-  Advantages: Improves user experience, prevents immediate inconsistency.
-  Disadvantages: Other clients may still see stale data, increases metadata storage requirements.
-  Example Systems: Social media apps, Google Drive, Dropbox.

Read-Your-Writes Consistency - Example

- Alice writes a **Facebook post at 10:00 AM**.
- If she refreshes at **10:01 AM**, she must see her post immediately.
- **Other users may still see stale data** due to delayed replication, but Alice won't.

Monotonic Reads – Preventing Time Travel in Reads

- Ensures that if a client reads an older version first, all future reads will be the same or newer.
- Prevents 'time-traveling reads' where a user sees newer data, then an older version later.
-  Advantages: Prevents erratic behavior, useful for caching and distributed reads.
-  Disadvantages: No guarantee of always seeing the latest write.
-  Example Systems: Newsfeeds, notification systems, cloud storage synchronization.

Monotonic reads - Example

- Alice logs into **Facebook at 9:00 AM** from her **phone** and sees Bob's latest post.
- At **9:05 AM**, she switches to her **laptop** and refreshes the page.
- **With monotonic reads**, She will **never see an older version of her feed after seeing a newer one.**

Global Consistency Models (Strong → Weak)

Model	Ensures	Use Cases	Guarantees for Clients
Strong Consistency	Latest Write Always Visible	Banking, Transactions	Every read gets the most recent write
Causal Consistency	Causal dependencies preserved	Chat systems, Collaborative editing, Social media	Related updates are ordered correctly
Eventual Consistency	Data converges over time	Social media, CDNs	Reads may return stale data



Client-Level Guarantees (Session Semantics)

Model	Ensures	Use Cases	Guarantees for Clients
Session Consistency	Each client sees its own latest writes	Mobile apps, Cloud services, E-commerce	A user never sees outdated writes
Monotonic Reads	No time-traveling reads	Caching, Cloud storage	Clients never see an older version after reading a newer one
Read-Your-Writes	Client sees its own updates immediately	User-generated content	A user always sees what they just wrote

Industry Trade-offs: Banking vs. Social Media

- Banking Systems: Require strong consistency (CP), no data loss.
- Social Media: Prefers high availability (AP), tolerates stale data.
- Example: Financial transactions vs. social media likes.

How Different Products Use Replication and Consistency

- Google Spanner (CP): Strong consistency, TrueTime API, 2PC.
- Amazon DynamoDB (AP): Eventual consistency, quorum-based replication.
- Apache Cassandra (Tunable): Adjustable consistency levels, quorum reads/writes.
- Firebase Firestore (Strong): Multi-master replication, real-time sync.
- Netflix CDN (Available): Content replication for low-latency streaming.